The start of the packet will always contain the ASCII character code for `2` and the end of the packet will be the ASCII code for `3`. We check for the start code by reading the first byte after serial is available and compare that to the value `2`. After the start byte, we will read the next 12 bytes that will contain one of 16 hexadecimal values including 0 through 9 and the letters A through F. These are added one byte at a time to the `tagString[]` array, incrementing `index` each time through the while loop. The remaining three characters, a newline character, carriage return, and the end byte, are unimportant for our purposes and are cleared from the buffer to prevent any problems later.

If everything has been read correctly, the function will return the boolean value `true`, which is then used to call the `printTag()` function. This function will print each of the 12 hexadecimal digits to our Serial Monitor so that we can see the value of the tag before printing a call to the `compareTags()` function. Assuming that we want to do more than just print the value of the tag read, it might be nice to compare that value to known tag IDs. Our last function uses the C function `strncmp()` (discussed later) to compare the incoming ID with a list of known IDs and will then return a text string or a name for that tag. Rather than returning the text `"Tag 1"` or `"Tag 2"`, we could as easily change the color of some lighting, print this information to a display, store the information somehow, or even open a door for a cat.

Most of the code should look familiar, even if it appears in a slightly different manner than you might have seen before. We will look at the specific functions of the SoftwareSerial library that are used in our example project that differ from the Serial functions discussed earlier.

# SoftwareSerial Library

While Listing 10-2 could work fine with the RFID reader connected to the RX pin, or pin 0 on the Arduino board, it would be a pain to have to disconnect the reader each time we want to upload a new sketch. Or we might have a second serial device that we need to talk to in addition to the RFID reader. In order to have more serial ports than what our hardware comes with, we can use a library that will create a software serial port for us that act almost like the hardware version. To create software-based serial ports, we need to use the SoftwareSerial library, an all-new library as of Arduino version 1.0, based on the previous library formally known as NewSoftSerial, written by Mikal Hart. This new library replaces the old library of the same name and brings with it many improvements and a couple of trade-offs. At the time of this writing, the best place to find out more about the SoftwareSerial library is at the NewSoftSerial web page at `http://arduiniana.org/libraries/newsoftserial`.

The SoftwareSerial library will allow us to emulate the serial RX and TX pins of a serial port on any of the available I/O pins on the Arduino Uno board. (This library may only work on certain pins on other versions of the Arduino hardware.) While this “virtual” serial port lacks the buffer memory like the native hardware serial (so it cannot receive data in the background), it does share most of the same functions that the Serial library contains, with similar syntax and functionality. The newest version of this library is interrupt driven, which means that it will conflict with hardware interrupts like we used in Chapter 7, as well as the newest Servo library, which is also interrupt driven. If we absolutely must use interrupts in our code, we would either need to use a different or older version of either the SoftwareSerial or Servo libraries. Finally, SoftwareSerial also slows things down a bit, so we should avoid excessively slow data rates and generally stick with hardware serial—unless we absolutely need the extra serial port.

So, let’s look at a few of the functions that make our project code work. If we don’t discuss most of the functions that we used, that’s because they work identically to those same functions in the Serial library.